Lesson 8 — What Each Notebook Shows

Notebook guide, lesson 8 — Technologies for Artificial Intelligence

Fabio Antonini — Università degli Studi dell'Aquila

20 November 2026 · three notebooks

Contents

Notebook 01 — `01_kmeans_by_hand` **1**
k-means by hand, and how many groups there are 1

Notebook 02 — `02_merge_trees_and_dbscan` **2**
When groups are not round: a merge tree, and DBSCAN 2

Notebook 03 — `03_pca_and_anomalies` **3**
Principal component analysis (PCA), and the anomalies a 2-D plot cannot show 3

A short guide to this lesson's notebooks. For each one: what it is for, the concepts in the order they appear, the numbers worth noticing, and **the two-minute version**, a summary to read before running it or to come back to afterwards. Every number here comes from the committed output of the notebook it belongs to.

Notebook 01 — `01_kmeans_by_hand`

k-means by hand, and how many groups there are

What it is for. The first notebook of the course with **no y anywhere**. Aurora, an online retailer, wants to group 2,000 customers that nobody has labelled.

The data. 2,000 customers, two columns (annual spend, visits per month), four true segments kept aside and never shown to any method.

The concepts, in order.

1. **Look first.** Four round clouds are visible before any algorithm runs.
2. **k-means in ten lines**: `assign` (each point to its nearest centre), `move` (each centre to the mean of its points), and `wcss`, the within-cluster sum of squares.
3. **Six customers on paper.** Starting both centres on A and B, the WCSS goes **220 → 56.25 → 16.6875 → 2.6667** and the algorithm stops at ABC and DEF (`kmeans_by_hand.png`).
4. **Two starts on the real data.** One start stops at WCSS **374.1**, another at **1,390.4** (`kmeans_two_starts.png`); over 30 starts, **29** reach 374.1. scikit-learn's `KMeans`, best of 10 starts, gives the same **374.0919**.

5. **How many groups?** The silhouette of customer B, worked by hand: $a = 1.207$, $b = 8.536$, $s = 0.859$ (`silhouette_one_point.png`). On the 2,000 customers the elbow and the silhouette both point to $k = 4$, where the silhouette peaks at **0.690** (`elbow_silhouette.png`).
6. **Against the truth.** Adjusted Rand index **0.985**, with the crosstab showing where the few disagreements are (`kmeans_vs_truth.png`).

The two-minute version. k-means is two moves repeated until nothing changes, and its score can only fall, so it always stops, though not always at the best grouping: run it from several starts and keep the best. k is your choice; the silhouette's maximum is the easiest guide. On four round clouds, k-means finds the true segments almost exactly.

Notebook 02 — `02_merge_trees_and_dbscan`

When groups are not round: a merge tree, and DBSCAN

What it is for. A dataset built so that k-means **cannot** solve it, a dense core of bots inside a ring of humans, and the method that can.

The data. 1,500 website sessions, duration and pages viewed; 180 bots (12%), with the same average as the humans and a much smaller spread.

The concepts, in order.

1. **k-means with** $k = 2$: ARI **−0.046**, all 180 bots in one cluster together with 780 humans. Its boundary is always a straight line.
2. **A tree of merges** on eight points: merges at heights 0.671, 0.894, 1.166, 1.275, 1.333, then a jump to 7.193; cutting below the jump gives three groups (`merge_tree.png`).
3. **Ward linkage on the sessions**: ARI **−0.062**, the same failure as k-means (`round_methods_fail.png`).
4. **DBSCAN drawn** on twelve points with `eps = 0.9`, `min_samples = 4`: 4 core, 4 border, 4 noise (`dbscan_points.png`).
5. **Choosing `eps`** with the k-distance plot: only **65** sessions have their 10th neighbour beyond 0.30 (`k_distance_plot.png`).
6. **DBSCAN on the sessions**: ARI **0.941**; all 180 bots in one group, 1,303 of 1,320 humans in the other, 4 humans with the bots, **13** noise sessions, all humans, 2 in the gap around the core and 11 on the ring's outer edge (`dbscan_result.png`).
7. **The silhouette prefers the wrong answer**: 0.401 for k-means' split, −0.106 for DBSCAN's correct one; **99.2%** of the ring's points have a negative silhouette.

The two-minute version. k-means and Ward both look for compact groups, so on a core inside a ring both cut the picture in half. DBSCAN groups by how crowded a neighbourhood is, needs no k, and is allowed to say "noise". The silhouette, the only score available without labels, rewards compactness and so preferred the wrong answer here.

Notebook 03 — `03_pca_and_anomalies`

Principal component analysis (PCA), and the anomalies a 2-D plot cannot show

What it is for. Finding how many numbers really vary in an 8-column table, and using that to catch fraudulent accounts that look ordinary on every column.

The data. 2,000 accounts, 8 columns driven by 3 hidden tendencies, with 40 planted anomalies.

The concepts, in order.

1. **Correlation**: annual spend and basket value **0.859** (`feature_correlation.png`).
2. **The widest shadow** on those two columns: variance **1.000** on the spend axis, **1.859** $= 1 + r$ at 45°, **0.141** $= 1 - r$ at 135° (`pca_shadow.png`). Checked against the eigenvalues of the covariance matrix and against `PCA`: shares **0.9297** and **0.0703**.
3. **Scree plot**: three components carry **93.5%** of the variance, the fourth **2.4%** (`pca_scree.png`).
4. **Every value ordinary**: 100% of the anomalies' values lie inside the genuine 5th–95th percentile range, yet they fall off the band (`anomalies_off_the_band.png`).
5. **Reconstruction on two columns**: a genuine account at 4.70 from the middle with error **0.01**; an anomaly at 1.96 with error **3.75** (`reconstruction_two_columns.png`).
6. **Reconstruction error on eight columns**: flagging the worst-rebuilt 2% catches **37 of 40**; mean error 4.676 for anomalies against 0.431 (`reconstruction_error.png`). Over eight batches: **32 to 38**, median 36.
7. **2-D pictures**: on PCA's first two components the anomalies sit **1.30** from the middle against **2.20** for genuine accounts; t-SNE shows the same (`pca_tsne_anomalies.png`).

The two-minute version. PCA finds the directions along which the data spreads most; three of them describe Aurora's eight columns. An account that every one of its columns calls ordinary can still break how the columns move together, and the kept components then rebuild it badly: that catches 37 of 40. Squashed into a 2-D picture, the same accounts sit near the middle, so looking for points at the edge finds the wrong ones.